



UNIVERSITÀ DI PARMA

Binary Images Morphological Processing

- Binary images
- Thresholding
- Mathematic Morphology
 - (also for non binary images)
- Connected components extraction

In queste slide definiremo cosa si intende per immagine binaria, cosa è l'operazione di "sogliatura" e introdurremo la morfologia matematica (estesa anche alle immagini non binarie)

Anche se, a rigore, non è solo per immagini binarie parleremo di estrazione di componenti connesse



UNIVERSITÀ DI PARMA

Binary Images



BINARY IMAGES

- In some cases it is necessary/useful to acquire/produce and/or process images whose pixels can only have 0/1 logical values
 - PBM or XBM image formats

```
00010010001000
00011110001000
00010010001000
```

Esistono tutta una serie di situazioni in cui ha senso avere immagini i cui pixel assumono solo due valori. Acceso e spento o bianco e nero.

A livello di codifica quindi assumeranno due valori e, in alcuni casi, questi sono 1 e 0 come ad esempio in alcuni formati grafici.

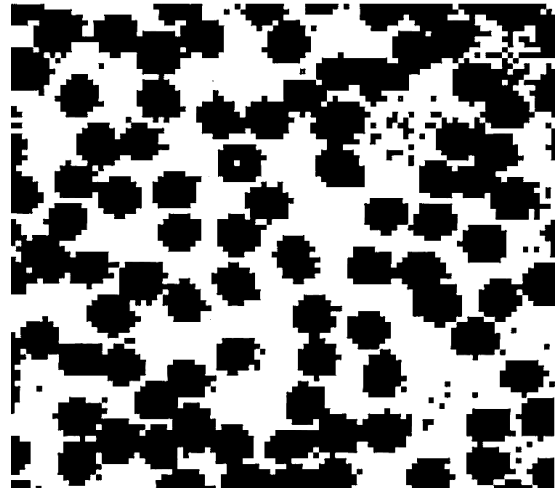
Tecnicamente potrei usare 1 bit per pixel. Pratico per risparmiare memoria ma in generale un po' scomodo (e comunque noi la memoria non la paghiamo no?)

- 1 actually can be considered as !=0
- 255 is often used for simplicity

0	0	0	255	0	0	255	0	0	0	255	0	0	0
0	0	0	255	255	255	255	0	0	0	255	0	0	0
0	0	0	255	0	0	255	0	0	0	255	0	0	0

Ecco perché spesso si usa 0 e 255 (il nero e il bianco di cui parlavo prima). È un po' più semplice da gestire. Ad esempio uso 1 byte per pixel (ma questo potevo farlo anche con 0 e 1) ma soprattutto riesco facilmente a visualizzare tali immagini (con 0 e 1 direi di no...)

- Why binary images?
 - They are simpler to process
 - Example: Image of blood cells
 - We would like to estimate red blood cells
 - Anyway they are not isolated (<50%)
 - How we can separate them?
 - Often output of other processing techniques
 - i.e. edge detection



Per alcuni compiti sicuramente risulta più semplice elaborare immagini binarie oppure può anche essere l'output di alcuni tipi di elaborazione (parleremo di edge detection)

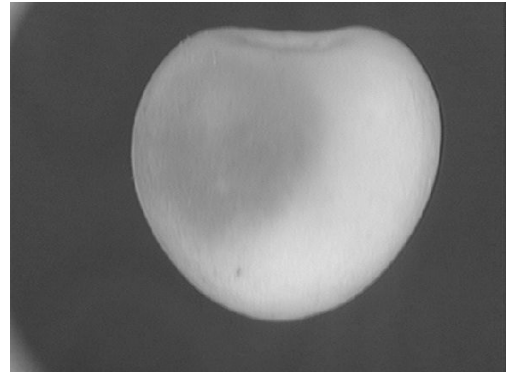
How to obtain binary images?

- Basically no B/W cameras
- Besides specific processing → thresholding
 - Pixel above a specific value → 1
 - Pixel below a specific value → 0

Tralasciando alcuni tipi di elaborazione come ottengo immagini binarie?

LA “sogliatura” è il tipico sistema, in pratica metto a 0 ciò che è al di sotto di un certo valore e 1 (o 255) ciò che è sopra.

- We would like to discard a bruised apple
- How we can select the bruised part?
 - Extract “dark” pixels
- We can only use pixel brightness
- Thresholding:

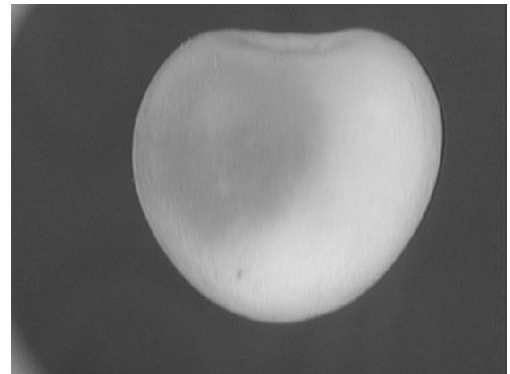


$$out(r,c) = \begin{cases} 0 & \text{where } in(r,c) < th \\ 1 & \text{where } in(r,c) \geq th \end{cases}$$

Approccio molto semplice che però nasconde un problema non banale, come scelgo la soglia th ?

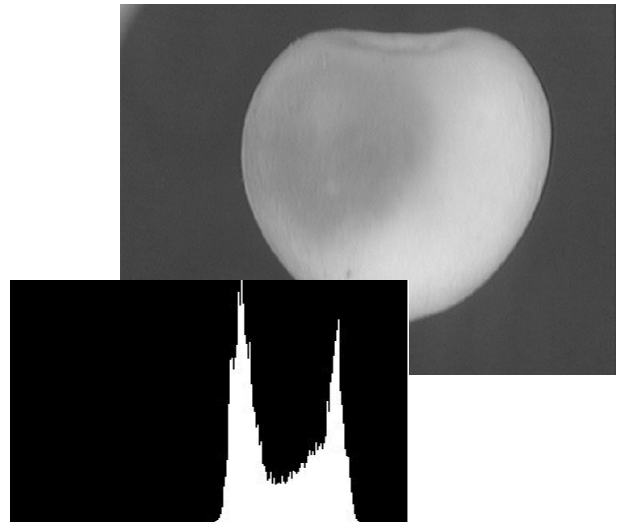
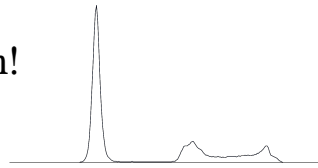


- Anyway we have:
 - “Good” apple part
 - Bruised apple part
 - Background
- How we decide about the threshold?



$$out(r,c) = \begin{cases} 0 & \text{where } in(r,c) < th \\ 1 & \text{where } in(r,c) \geq th \end{cases}$$

- Anyway we have:
 - “Good” apple part
 - Bruised apple part
 - Background
- How we decide about the threshold?
- Histogram!



Vi ricordate cosa è un Istogramma dei toni di grigio?

Vettore di interi di dimensione pari a tutti i possibili livelli di quantizzazione dell'immagine

La dimensione del vettore istogramma non dipende dal contenuto della particolare immagine, ma solo dalla codifica. Es. immagine singolo canale 8 bit per pixel: 256

Nel caso in esame si notano 3 picchi, il più grande è in realtà relativo allo sfondo e quindi, dato l'obiettivo proposto, possiamo ignorarlo e usare solo l'istogramma che non contiene quella parte (quello con lo sfondo nero).

Presenta due picchi. Ci sta che uno corrisponda alla parte “buona” della mela e uno alla parte ammaccata.

La soglia per separare, sicuramente va messa tra i due picchi.

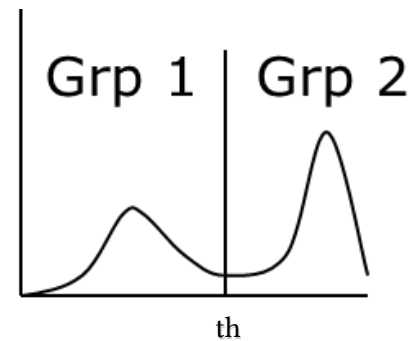
Ma esattamente dove se voglio la separazione ottima?

- Best threshold computation
- Problem: find a th that minimize weighted sum of each group variance

$$\sigma^2(th) = \sigma_1^2(th) \cdot w_1(th) + \sigma_2^2(th) \cdot w_2(th)$$

- $w_1(th)$ & $w_2(th)$ are the probability of belonging to group 1 or 2

$$w_1(th) = \sum_{i=0}^{th-1} h(i) \quad w_2(th) = \sum_{i=th}^L h(i)$$



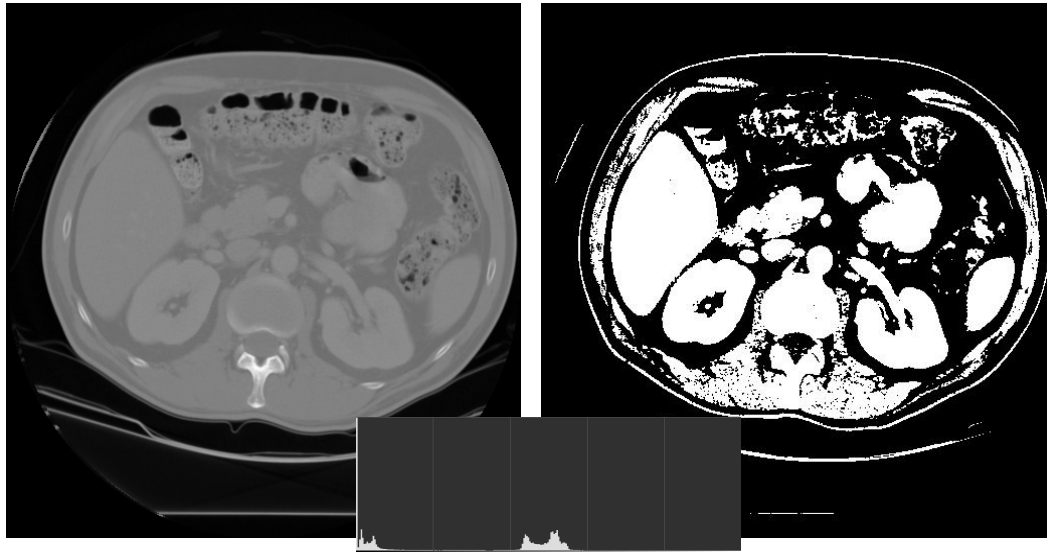
Nel caso di istogrammi come questo in cui ho due picchi significativi. Il metodo migliore è quello di Otsu. Data una soglia th io separo l'istogramma in due parti.

Di ciascuna di queste due parti io posso calcolare le varianze (il quadrato delle deviazioni standard σ_1 e σ_2). Calcolo la loro somma pesata con la probabilità di far parte del relativo gruppo (l'integrale discreto dell'istogramma prima e dopo th).

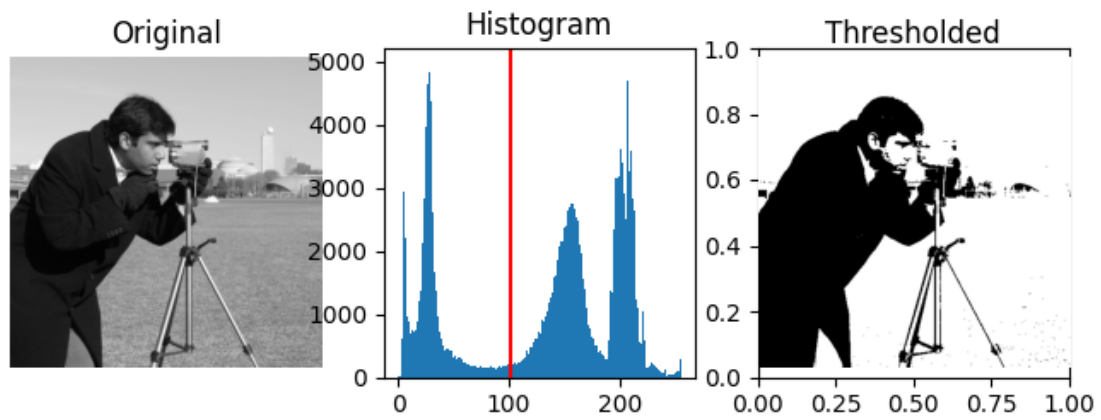
Il th che minimizza questa somma è la soglia ottima. In pratica minimizza la varianza intraclasse.

Funziona bene se e solo se ho due classi (come qui dove abbiamo ignorato lo sfondo o in situazioni specifiche tipo OCR)

Nobuyuki Otsu, A threshold selection method from gray-level histograms, in IEEE Trans. Sys., Man., Cyber., vol. 9, 1979, pp. 62–66, DOI:10.1109/TSMC.1979.4310076.



Situazione simile alla mela, sfondo a parte è abbastanza bipolare e il risultato non è male



Qui non è bipolare ma quasi, risultato accettabile



Costei non è decisamente bimodale... e il risultato non è un granché



UNIVERSITÀ DI PARMA

Morphological Processing

Morphological Processing



- Used to extract image components that are useful in the representation and description of region shape, such as
 - boundaries extraction
 - skeletons
 - convex hull
 - morphological filtering
 - thinning
 - pruning

LA morfologia matematica nasce per essere applicata ad immagini binarie (ma è stata estesa anche a immagini non binarie) e permette di modificare queste immagini ottenendo ad esempio informazioni come i bordi, lo scheletro o semplicemente preparandole per elaborazioni successive



- Main operations
 - Erosion $\rightarrow \ominus$
 - Dilation $\rightarrow \oplus$
 - Opening $\rightarrow \circ$
 - Closing $\rightarrow \bullet$
 - Hit or Miss $\rightarrow \otimes$

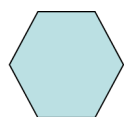
Le due operazioni principali sono l'erosione e l'espansione. Apertura e chiusura sono due operazioni che ottengo combinando le precedenti

- Small “mask” to probe the image under study
 - Convolution like approach
- Structuring Element features an origin
- Shape and size must be adapted to geometric properties we would like to mask/obtain



box

box(length,width)

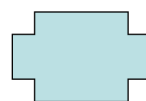


hexagon



disk

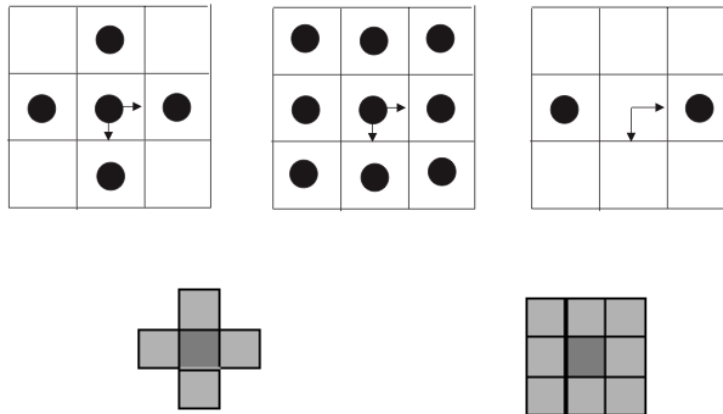
disk(diameter)



something

Il meccanismo è simile a quello visto per la convoluzione ma anche per altre operazioni non lineari. Ho una maschera (l'elemento strutturante) che posiziono sull'immagine originale “centrando” ciascun pixel. Come nel caso della convoluzione le operazioni che faccio riguardano i “pixel” dell'elemento strutturante e quelli sovrapposti dell'immagine. In generale sono operazioni “logiche”

A differenza della convoluzione gli elementi strutturanti hanno una origine (è questa che sovrappongo al pixel in esame) e possono anche avere forme disparate.



Esempi di elementi strutturanti.

Come vedete tutti hanno una origine (che a seconda degli autori può essere indicata in modi differenti).

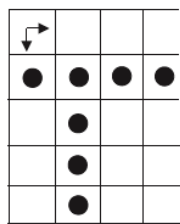
La forma è molto libera. Focalizziamoci ad esempio sull'elemento in alto a sx. Non è un quadrato ma una vera e propria croce!

- The structuring element B is “moved” (B_z) on the image A for a set of values z belonging to an Euclidean Space E
- The result is the set of z values for which B_z is in A

$$A \ominus B = \left\{ z \in E^2 : B_z \subseteq A \right\}$$

- Practically the structuring element is superimposed on all the pixel of the image.
 - When it completely fits the binary image the result is 1, otherwise 0
 - Logical AND
 - It shrinks image elements

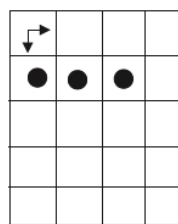
Erosione. Sovrappongo la maschera a ciascun pixel dell'immagine originale. In pratica faccio un AND tra tutti i pixel dell'immagine che sono sovrapposti ad un “pixel” dell'elemento strutturante.
La pagina dopo è illuminante



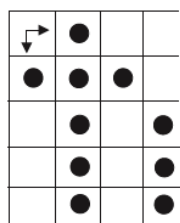
A



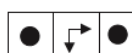
B



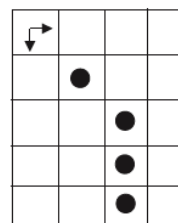
$A \ominus B$



A



B



$A \ominus B$

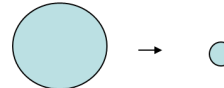
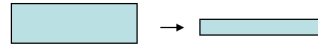
Dove riesco a sovrapporre B su A di modo che a tutti i “pallini” di B corrisponda un pixel “acceso” (anche qui rappresentato con un pallino) di A? Lo vediamo nelle due immagini a destra

Attenzione che questi due esempi sono per farvi capire il meccanismo. Questi due elementi strutturanti sono un po' inconsueti. Posso capire la croce come visto un paio di slide prima oppure un quadratino pieno, altri elementi sono raramente usati (ma chiaramente dipende dal problema)

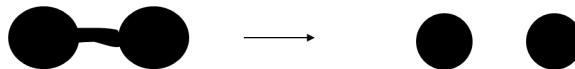
Erosion **shrinks** the connected sets of 1s of a binary image.

It can be used for

1. shrinking features



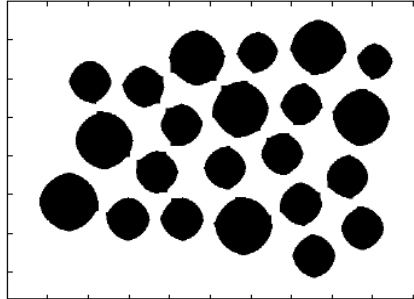
2. Removing bridges, branches and small protrusions



Dato che ho un AND logico mi basta anche un solo pixel “spento” sovrapposto alla maschera che otterrò un pixel spento nel risultato. Di fatto quando la maschera è sovrapposta a zone di “bordo” (delle sagome non dell’immagine), io azzerò quei pixel. Da cui il nome erosione.

Quindi riduco aree contigue oppure rimuovo piccoli elementi fini.

- Consider the following image



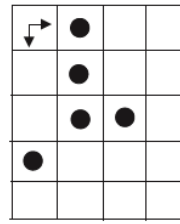
- We can use erosion to separate elements

Nel PDF non si vede l'animazione ma qui i vari elementi erano tutti uniti. Grazie ad una erosione con un semplice quadrato 3x3 li riesco a separare. Penso che adesso sia molto più semplice contarli.

- Again the structuring element is superimposed on all the pixel of the image.
 - When it hits the binary image the result is 1, otherwise 0
 - Logical OR
- It “dilates” image elements

L’Espansione (e non dilazione...) è il duale, il meccanismo è lo stesso ma faccio un OR

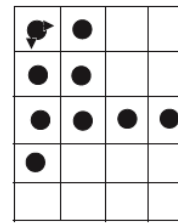
Dilation $\oplus$



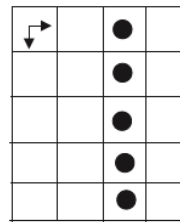
A



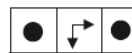
B



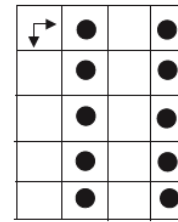
$A \oplus B$



A



B



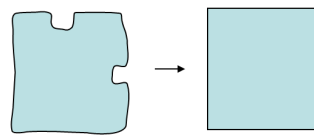
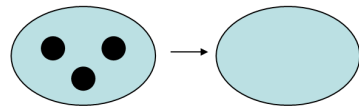
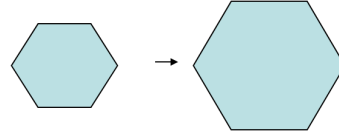
$A \oplus B$

Nell'esempio in basso supponiamo che l'elemento strutturante possa "sforare"

Dilation **expands** the connected sets of 1s of a binary image.

It can be used for

1. growing features
2. filling holes and gaps



L'uso è chiaramente opposto. Voglio far crescere le sagome o riempire piccoli buchini o pezzi che mancano

- We can use erosion and dilation to remove small details or to fill some gaps
- But they also affect other part of the image...
- Solution
 - Combine them!

Una mi permette di togliere elementi fini, l'altra riempie parti mancanti.
Cosa succede se le combino?

- Opening
 - Erosion followed by dilation

$$A \circ B = (A \ominus B) \oplus B$$

- Closing
 - Dilation followed by erosion

$$A \cdot B = (A \oplus B) \ominus B$$

- The same structuring element is used

Apertura e chiusura sono ottenute applicando prima un'erosione e poi un'espansione o viceversa

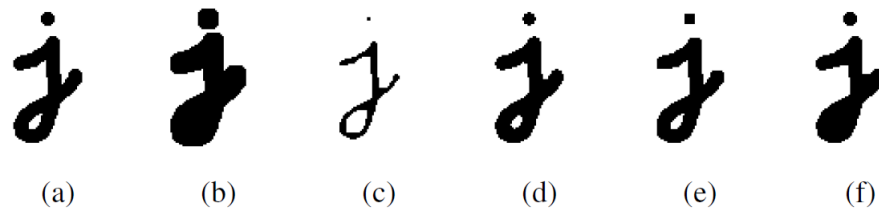


Figure 3.21 Binary image morphology: (a) original image; (b) dilation; (c) erosion; (d) majority; (e) opening; (f) closing. The structuring element for all examples is a 5×5 square. The effects of majority are a subtle rounding of sharp corners. Opening fails to eliminate the dot, since it is not wide enough.

Source: Szeliski

Esempi di applicazione di erosione, espansione, apertura e chiusura.

Il “majority” applica un “vince la maggioranza”, lo posso vedere come una erosione molto all’acqua di rose

- Two “disjoint” structuring elements are used: A e B

- $A \cap B = \emptyset$

- Two erosion are used and joined

- $I \otimes X = (I \ominus A) \cap (I^c \ominus B)$

- Where I^c is the complement for I and $X = (A, B)$

	1	
0	1	1
0	0	

- The idea is that A should match shapes and B background

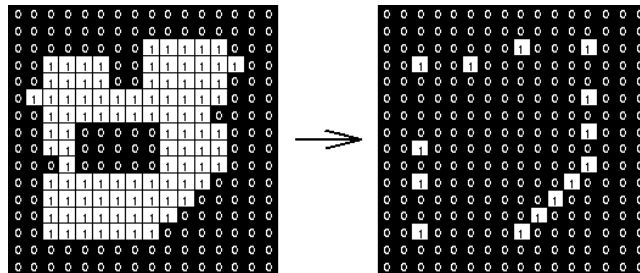
- It can be seen as an “extended” SE

Fino ad ora abbiamo visto elementi strutturanti in cui consideravamo solo i punti “accesi”. Nell’hit or miss si considerano anche pixel “spenti” come fossere un secondo elemento strutturante

L’idea è usare i secondi per fare match con i pixel spenti dell’immagine binaria usando nuovamente un AND logico.

Unisco questo secondo AND al primo o meglio considero la loro intersezione (che poi di fatto è un nuovo AND)

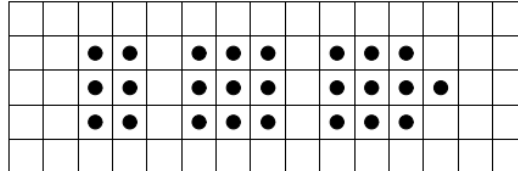
	1				1				0	0		0	0				
0	1	1		1	1	0		1	1	0		0	1	1			
0	0				0	0			1				1				



Dove nell'immagine originale “matcheranno” sia i punti 1 che gli 0? Negli angoli...

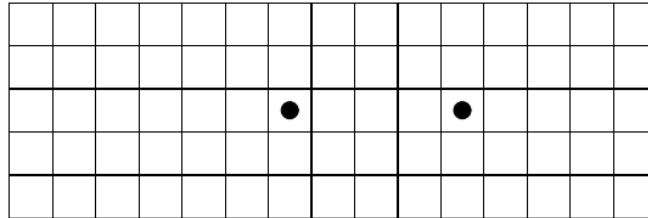


- Hit or Miss can be effectively used for pattern matching
- Let us locate a 3x3 square in the following image





- If we use a 3x3 square SE





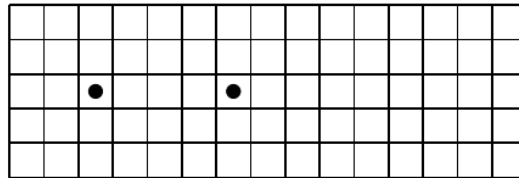
- If we use a 5x5 square border SE

•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
•	•			•				•				•	•	•
•	•			•				•					•	•
•	•			•				•				•	•	•
•	•	•	•	•	•	•	•	•	•	•	•	•	•	•

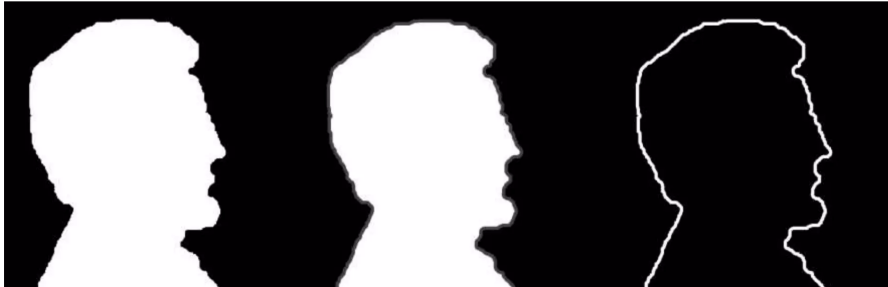
C:

•	•	•	•	•
•				•
•				•
•				•
•	•	•	•	•

- The result is:



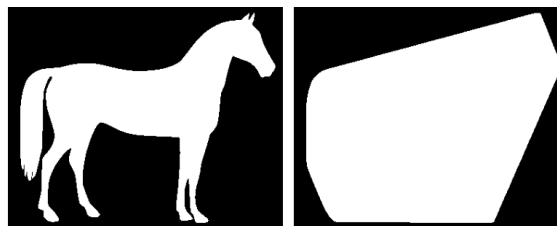
- Erode image using a given SE
- The difference between original image and eroded one is the “border”
- The thickness of the contour depends on the SE size



Con un elemento strutturante “pieno” (anche un banale quadrato 3x3) io elimino una fetta di pixel di bordo. LA differenza tra immagine originale e risultante è proprio il bordo stesso (più o meno spesso a seconda dell’elemento strutturante scelto)



- A set of points X is defined as **convex set** if any straight line segment between two X points completely lies inside X
- The **convex hull** H of a S set is the smallest convex set that contains S
- **Convex deficiency** is $H-S$



Data una sagoma il suo convex hull è la sagoma convessa più piccola che la contiene

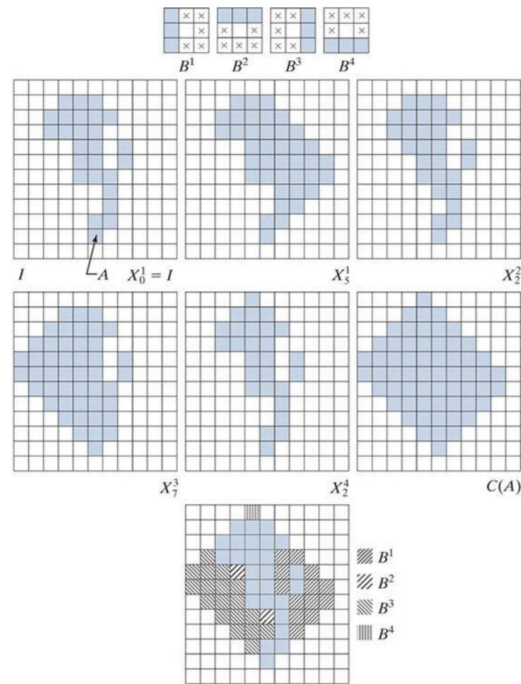
- Convex Hull can be computed using a set of hit and miss transform
 - Apply each SE to the original shape until the result is different
 - The union of the 4 resulting shapes is a convex hull



Usando questi 4 elementi strutturanti di seguito e ripetendo l'operazione ottengo una sagoma convessa

In realtà non è esattamente un convex hull in quanto spesso non è la più piccola sagoma convessa ma ci vuol poco a rimuovere le parti non necessarie

Not optimized!



L'immagine originale è quella indicata con I . Proviamo ad applicare una git or miss piú volte usando l'elemento strutturante B_1 . E ripetiamo l'applicazione di questo elemento fino a che ottengo una immagine risultante differente. Dopo 5 applicazioni ottengo l'immagine indicata come X^1_5 , se provo a ripetere nulla cambia e quindi mi fermo.

Usando gli altri 3 elementi strutturanti ottengo le altre X_{qualcosa} .

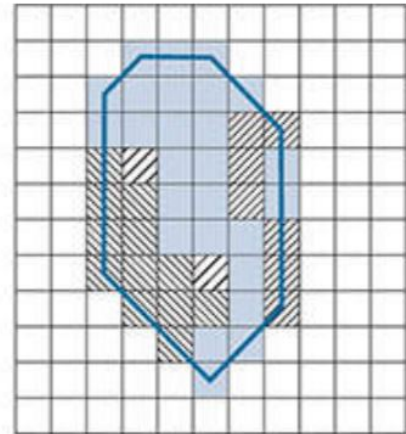
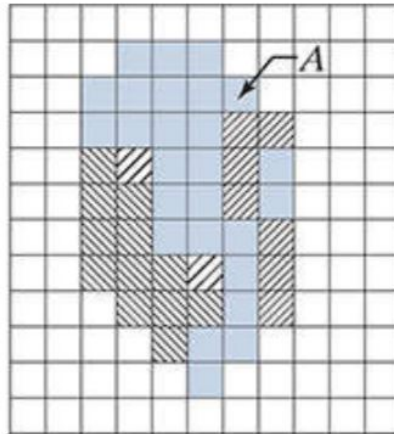
Unisco i 4 risultati e ottengo $C(A)$

Attenzione, è sicuramente una sagoma convessa che contiene quella originale.

Non è la piú piccola ottenibile. Ad esempio il pixel nella prima riga lo posso eliminare così come molti pixel sia a destra che sinistra. Ma è facile farlo, prendo i pixel di questa sagoma che erano accesi anche in quella originale e che si trovano sul bordo, traccio delle rette tra quelli consecutivi ed elimino ciò che sta "fuori"

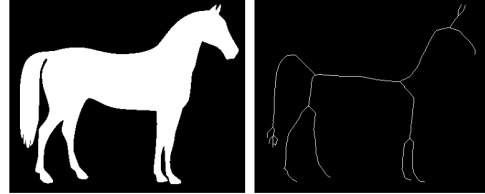


Edge pixels can be
used to “trim” the
result
Namely, limiting
the convergence in
the horizontal or
vertical directions





- Skeleton is a thin version of a shape
 - Single pixel thickness
 - Equidistant to shape borders
 - Topological equivalence
- Again a hit and miss approach
 - Up to 8 SEs





- Other morphological complex operations can be used using union/intersection:
 - Border extraction
 - Skeletonization
 - Thinning
 - ...





UNIVERSITÀ DI PARMA

Morphological Processing for Grey Level Images

Morphological Processing for Grey Level Images



- Mathematical morphology can be extended to grayscale images
- Applications
 - Contrast enhancement
 - Texture description
 - Edge detection
 - Thresholding

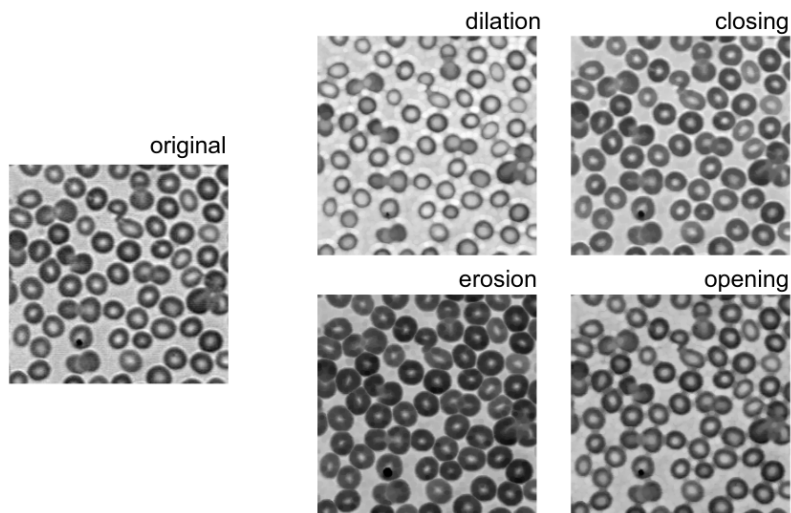
È possibile una estensione alle immagini a toni di grigio.

L'elemento strutturante è sempre binario.

L'immagine no, quindi non più operazioni logiche

- In a grayscale image there is no “object” and “background”
- Structuring element is still a mask
- Min and Max operators are used
 - Dilation $\rightarrow$ max element under the mask wins
 - Erosion $\rightarrow$ min element under the mask wins
- Opening & Closing as before

Di fatto uso il minimo o il massimo valore (limitiamoci alle immagini a toni di grigio) sovrapposto alla maschera.



Attenzione che i robbi rappresentati sono scuri e quindi è normale che diminuiscano di dimensione nell'espansione e aumentino nel caso dell'erosione



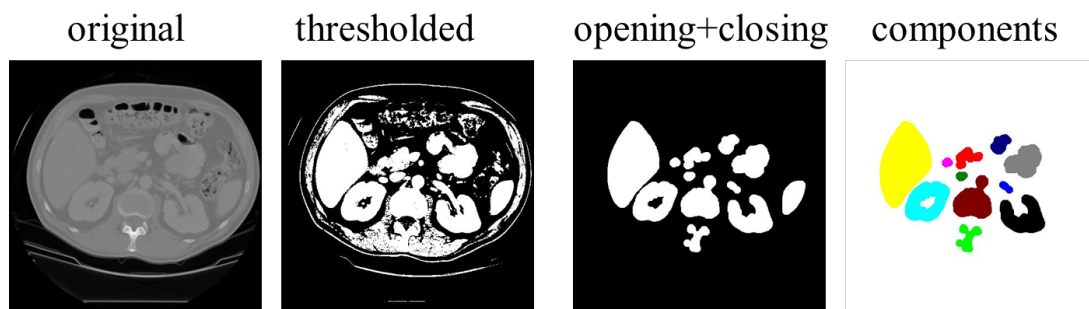
UNIVERSITÀ DI PARMA

Connected Components



Connected Components

- A connected component is a **contiguous** group of pixels that have the **same value**
 - Actually not only for binary images

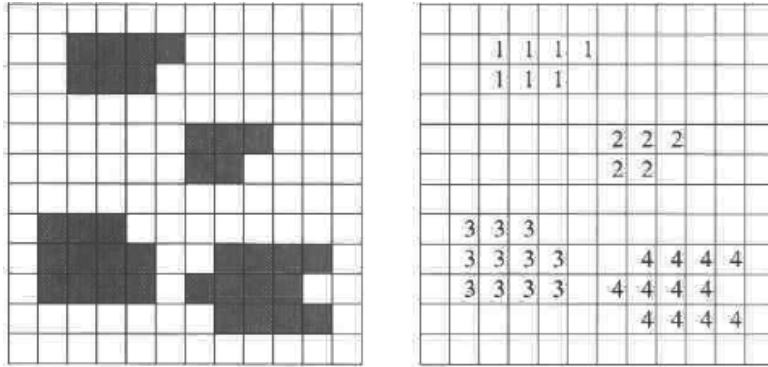


Abbiamo sogliaato l'immagine, poi con un po' di morfologia abbiamo isolato i vari pezzi adesso vogliamo etichettarli.

A rigore vale anche per immagini non binarie ma di solito si usa in immagini in cui i pixel assumono solo un ridotto insieme di valori



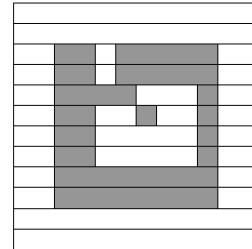
- We assign a unique label to each component



Source: R. Jain

Questo è il risultato che vorremmo...

- Different approaches can be used:
 - Recursive Tracking
 - Parallel Growing
 - Row-by-Row (widely used)



Ci sono diversi approcci, uno dei più semplici è l'ultimo e quindi mostriamo questo che necessita di due passi

- 1st pass:

- First row:

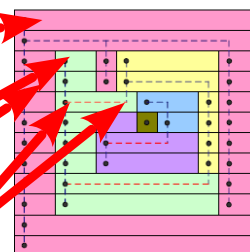
- Check left neighbor, do it have the same value?

- Yes → same label
 - No → new label

- Other rows

- Check left and upper neighbor

- They have same value and same label → same label
 - Current pixel have same value of only one of them → same label of that one
 - They have same value but different label (!) → set lowest label and track the equivalence
 - Otherwise → new label



Durante il primo passo io scandisco l'immagine riga per riga da sinistra a destra e, finita la riga, dall'alto verso il basso ripartendo da sinistra

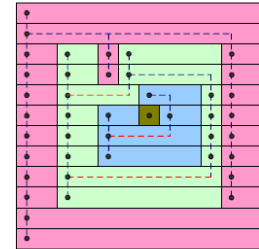
Inizializzo una label ad uno specifico valore (il rosino nell'immagine)

Per la prima riga, al primo pixel assegno quella label. Per tutti i successivi pixel che incontro li confronto con quello alla loro sx. Sono uguali? Sì → stessa label (come capita sempre in questo esempio), no → label differente (ma qui non capita)

Per le righe successive opero in maniera simile ma non solo confronto il pixel in esame con quello a sx ma anche con quello sopra. Posso avere differenti casi:

1. hanno tutti e 3 lo stesso valore e stessa label. Facile, gli assegno tale label (ad esempio i primi pixel delle colonne a sx)
2. il pixel in esame ha lo stesso valore di solo uno dei due (va da sé che questi abbiano anche label differenti) → prendo la label del pixel corrispondente (ad esempio quarta riga il pixel con la label "giallo" tutto a sx)
3. il pixel in esame ha lo stesso valore di entrambi i pixel ma loro hanno label differenti tra di loro(!). Esempio: pixel verde tutto a dx nella quinta riga. Gli associo la label più bassa e mi segno da qualche parte che le due label in questione devono però essere la stessa
4. nessuno dei due pixel ha valore uguale a quello in esame → nuova label ottenuta incrementando la precedente

- 2nd pass:
 - During 1st pass we have tracked label equivalences
 - Row by row change labels with lowest equivalent

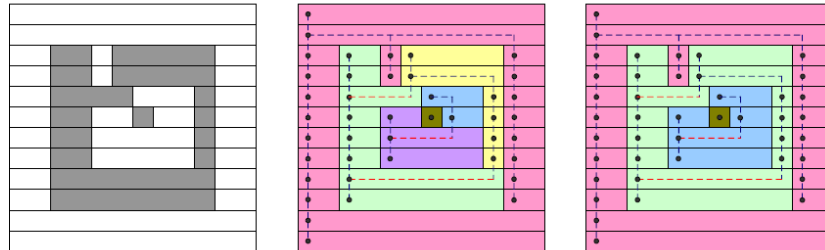


Durante il secondo passo uso il fatto che avevo memorizzato l'equivalenza tra alcune label originariamente assegnate (ad esempio giallo e verde). Riscandisco l'immagine e sostituisco le label di modo che ce ne sia una sola di quelle equivalenti.



- Row by Row

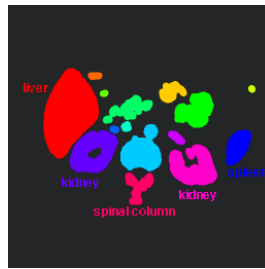
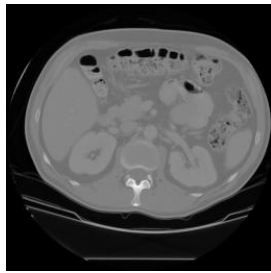
Source: Szeliski



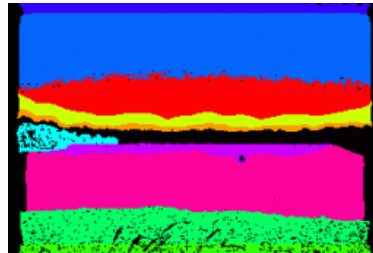
Per come è fatto l'algoritmo qui si intendono come zone connesse solo quelle che sono connesse in orizzontale o verticale, ma non diagonale. Da qui la presenza del quadratino marroncino.

Però l'algoritmo è facilmente estendibile a considerare anche la contiguità in diagonale.

Connected Components



connected
components
of 1's from
cleaned,
thresholded
image



connected
components
of cluster
labels

Con le immagini non binarie non funziona un gran ch  anche se clusterizzo i valori